

Nombre y Apellido:..... Curso: Profesor:

TEORÍA - SC							TEORIA - D			PRÁCTICA				
1	2	3	4	5	6	7	1	2	3	1	2	3	4	5

Teoría - Single choice: elija para cada pregunta la opción correcta (solo una es válida)

- 1- Una instrucción privilegiada puede ser ejecutada:
a) En modo usuario b) En modo kernel c) En ambos modos d) No se pueden ejecutar
- 2- Cuando un proceso crea a otro proceso, necesita:
a) Disparar una interrupción b) Ejecutar una instrucción privilegiada c) Ejecutar una syscall d) No puede crear otro proceso
- 3- ¿Desde cual estado NO se puede pasar a ready?
a) New b) Run c) Bloqueado d) Bloqueado / Suspendido
- 4- Los hilos de un mismo proceso comparten:
a) Heap b) Stack / Pila c) Registros d) Ninguna es correcta
- 5- ¿Cuál de estos algoritmos impide la monopolización de la CPU?
a) HRRN b) Prioridades sin desalojo c) FIFO d) VRR
- 6- La mutua exclusión sin espera activa, puede lograrse:
a) Con la Instrucción test_and_set b) Deshabilitando interrupciones c) Software / Peterson d) No es posible
- 7- La técnica para asegurar la no ocurrencia deadlock con bajo overhead y alta flexibilidad para la solicitud de recursos sería:
a) Evasión b) Detección y Recupero c) Prevención d) Ninguna es correcta

Teoría - Desarrollar: responda las siguientes preguntas justificando sus respuestas

- 1 – Compare las arquitecturas monolíticas y microkernel en términos de mantenibilidad, overhead y tolerancia a fallas.
- 2 – ¿Si dos hilos manipulan la misma variable, que problema puede ocurrir? Brinde un ejemplo de pseudo-código explicando en detalle cómo ocurría el problema.
- 3 – Sobre la Prevención de Deadlock, indique en qué consiste. Explique tres maneras en la que podría implementarse. Qué ventajas o desventajas puede tener con respecto a otras estrategias de tratamiento de Deadlock implementadas por el Sistema Operativo con respecto al uso de recursos.

Práctica - Desarrollar: resuelva los siguientes ejercicios justificando sus respuestas

1 - Se dispone de un sistema con planificador RR con Q=3, donde hay tres procesos activos; PA, PB y PC. PA inicia en el instante 10 una ráfaga de cpu de 4 ut. PB inició una ráfaga de I/O de 4 ut, en el instante 9. PC llega al sistema en el instante 13. Indique, para el instante 13:

- ¿Quién continúa ejecutando luego de ese instante?
- ¿Quienes quedan en Ready y en cual orden?
- ¿Ocurrieron interrupciones / syscalls? ¿Cuáles?.
- Si el algoritmo fuese VRR Q=3 ¿Puede asegurar que la respuesta de punto a) sería la misma?

2 - En una computadora con 2 CPUs, se tiene un planificador a nivel SO de tipo FIFO, mientras que los ULTs planifican SJF sin desalojo. Dada la siguiente traza:

		Llegada	CPU	IO
KLTA	ULT1	0	3	-
	ULT2	0	2	-
KLTB	ULT3	2	1	2
	ULT4	2	2	-

Grafique el diagrama de gantt para reflejar la ejecución de dichos hilos.

3- Se dispone del siguiente pseudo-código, utilizado para dos procesos que se envían mensajes a través de una lista, cuyo máximo de elementos es 10. Existe una variable global llamada LISTA;

Proceso 1 (N instancias)	Proceso 2 (N instancias)
wait(C) wait(B) enviar_mensaje(LISTA) signal(B) signal(A)	wait(A) wait(B) recibir_mensaje(LISTA) signal(B) signal(C)

Asumiendo que el pseudo código es correcto, inicialice los semáforos para cumplir con las pautas establecidas.

4- En un sistema que utiliza detección y recuperación de deadlock, ejecutan dos KLT. El algoritmo de planificación utilizado por el sistema operativo es RR Q=3. El algoritmo de detección y recuperación, el cual ejecuta como un Proceso más, se bloquea, luego de ejecutar por 5 unidades de tiempo (ut) para volver a ejecutar según la planificación. Además, soluciona el deadlock finalizando procesos con mayor tiempo restante de ejecución.

Grafique el diagrama correspondiente a la traza de ejecución hasta el instante 25 e indique en qué instante finalizó cada KLT.

PROCESOS	KLTA	KLTB	Detector de Deadlock	-a y b: variables globales -d: variable local - deadlock_detect() : 2 ut -get(recurso): 2 ut -release(recurso): 1 ut -sleep(n): 0 ut -operaciones con variables: 1 ut -print(): 2ut
TIEMPO DE LLEGADA	1	2	0	
	get(recurso_A) a++ get(recurso_B) b=a+b d=b*2 release(recurso_B) release(recurso_A) print(d)	get(recurso_B) b++ get(recurso_A) b=a+b release(recurso_A) release(recurso_B)	while(true) { deadlock_detect() sleep (5 ut) } 	

NOTA: get(recurso) es bloqueante si el recurso no está disponible. Los recursos A y B sólo tienen una instancia cada uno.

5-

Dado un SO con planificación RR con q=2, donde la implementación de las funciones de semáforos es a través de la deshabilitación de interrupciones:

- Grafique el diagrama de gantt** de los siguientes KLTs.
- Indique el nivel de multiprogramación y el estado de la lista de Blocked en los instances 1, 4 y 8
- Realice un grafo de asignación de recursos para el instante 4

Nota: el semáforo S está inicializado en 1

KLTA (lleg: 0)	KLTB (lleg: 1)	KLTC (lleg: 2)
wait(S) // 3 ut CONT++ // 2 ut signal(S) // 3 ut	wait(S) // 3 ut CONT++ // 2 ut signal(S) // 3 ut	CONT++ // 2 ut

Criterio mínimo de aprobación: 4 teóricas SC correctas, 1,5 teóricas a desarrollar correctas, 3 ejercicios correctos.